Predicting Coronary Heart Disease from the BRFSS Survey

School of Computer and Communication Sciences, EPFL, Switzerland

***Abstract*—**

I. Introduction

II. Gradient-Boosted Decision Trees

The idea of GBDT [4] is to build an additive model on the log odds, $s(\mathbf{x}) = s_0 + \eta \sum_{k=1}^{K} f_k(\mathbf{x})$, where $f_k(\mathbf{x})$ represents k-th tree, and s_0 represents base rate of the positive class, that is:

$$s_0 = \log \frac{\hat{p}}{1-\hat{p}}, \quad \hat{p} = \frac{1}{n} \sum_i \mathbf{1}\{y_i = 1\}.$$

The goal of the method is to have a sequence of trees that would sequentially improve the scores. At each round, we compute for every single sample the gradient $g = p - y$, where $p = \sigma(s(\mathbf{x}))$ and the Hessian $h = p(1-p)$, and fit one regression tree to all of them. The intuition is as follows: Suppose that we have $y = 1$ and current prediction probability $p = 0.3$, it's clear to see that the gradient will be given by $g = -0.7$ which means that we should increase the score. Similarly, the Hessian represents the curvature, or the confidence in our prediction, as we can see that the expression will be maximized as $p \to \frac{1}{2}$, and is minimized for confident predictions $p \to \{0, 1\}$.

At each leaf we take a Newton-step $-\frac{G_\ell}{H_\ell + \lambda_{\text{leaf}}}$, where G_ℓ and H_ℓ are the sums of g and h over that leaf. This is the result of minimizing the second-order Taylor expansion over parameter w of the expression:

$$\sum_i \ell(y_i, s_i + w) \approx \text{const} + G\, w + \tfrac{1}{2}(H + \lambda)\, w^2,$$

where $G = \sum g_i,\ H = \sum h_i$ [5].

Each tree outputs w_{leaf} for all the samples in it. We can see that splitting a node into left and right children gives us the following advantage:

$$\text{gain} = \tfrac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right],$$

where L and R indicate left and right child, respectively. We only split a node if both children have $H_L, H_R \geq H_{min}$, so that we do not create trees for small number of already confident samples.

Finally, since finding the splitting thresholds would be too slow if we took into account all the possible thresholds, we decide on binning our values inside 64 bins, where bin 0 corresponds to NaN values, while the other 63 bins are computed on the train set and are ordered. For ordered values, the bins smaller than the threshold go to the left child and the larger ones to the right, while the 0-th bin can go either way. For categorical bins, we use the trick from [6] so that we overcome combinatorial nature of all possible permutations.

We fit each tree on 70% random rows and columns (features), with learning rate $\eta = 0.05$, $H_{min} = 10$, and regularization parameter $\lambda_{\text{leaf}} = 1$. All randomness is seeded, so the submission is exactly reproducible.

A. Results

To test the GBDT model we use 5-fold CV. To determine the number of trees we use early stopping with patience 50 epochs, on 10% hold-out of the current training fold, which also serves to choose the F1-maximizing decision threshold. Since this hold-out contains only ≈2,300 positives, the threshold is noisy (0.20–0.25 across folds); for the final model we therefore also choose it on the pooled out-of-fold predictions of the other four folds, which never uses the scored fold. Table I summarizes the results.

For features, we started with 97 processed features which were first used in the linear model, where GBDT achieves F1 of 0.4403, already greater than all logistic variants we previously tried. Additionally, giving it all 176 recoded columns from preprocessing with NaNs kept (NaNs are in the previously mentioned 0-th bin.) increases F1 by $+0.002$ and lowers average log-loss $0.2190 \to 0.2181$. With the threshold from the other folds, the best value is $\tau = 0.21$ in every fold and we achieve F1 of 0.4435 ± 0.0038 across 5 folds using τ.

Regarding hyperparameters, we do small search on fold 0. We find that depth 3 is too shallow for different interactions, and depths 4/5 are very close to each other (these results hold for learning rate $\eta = 0.1$). Lowering $\eta = 0.05$, changes the picture such that now depth 4 achieves the lowest log-loss, and for this depth the folds stop at 464 trees on average.

For this model (depth 4, $\eta = 0.1$) adding raw leftover columns decreases log-loss by no more than 10^{-4}, with F1 difference around ± 0.002, which are not always followed by the sign of log-loss difference.

The final model trained on the full training set is constructed with 500 trees (to account for an increase in data) and achieves **F1 0.451** and **Accuracy 0.877**, for threshold $\tau = 0.21$.

References

[1] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM Journal on Imaging Sciences*, vol. 2, no. 1, pp. 183–202, 2009.

Table I
GBDT RESULTS. TOP: 5-FOLD CV (MEAN) BOTTOM: FOLD 0, LOG-LOSS $\times 10^3$.

5-fold CV	F1	Log-loss
97 imputed features	0.4403	0.2190
176 recoded, NaN kept	0.4423	0.2181
+ threshold $\tau = 0.21$	**0.4435**	0.2181
Fold 0	$\eta = 0.1$	$\eta = 0.05$
depth 3	216.85	–
depth 4	216.69	**216.24**
depth 5	216.63	216.32
depth 4 + dropped raw cols.	216.62–217.34	–

[2] B. O'Donoghue and E. Candès, "Adaptive restart for accelerated gradient schemes," *Foundations of Computational Mathematics*, vol. 15, no. 3, pp. 715–732, 2015.

[3] N. Meinshausen, "Relaxed lasso," *Computational Statistics & Data Analysis*, vol. 52, no. 1, pp. 374–393, 2007.

[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017.